

Chapter 3

Low-Power CPU

3.1 Overview

The ESP32-P4 Low-Power CPU (LP CPU) is a 32-bit processor based upon RISC-V ISA comprising integer (I), multiplication/division (M), atomic (A), and compressed (C) standard extensions. It features ultra-low power consumption and has a 2-stage, in-order, and scalar pipeline. The LP CPU core complex has an interrupt controller (INTC), a debug module (DM), and system bus (SYS BUS) interfaces for memory and peripheral access.

The LP CPU is in sleep mode by default (see Section 3.10). It can stay powered on when the chip enters Deep-sleep mode (see Chapter 14 *Low-Power Management* for details) and can access most peripherals and memories (see Chapter 7 *System and Memory* for details). It has two application scenarios:

- Power insensitive scenario: When the High-Performance CPU (HP CPU) is active, the LP CPU can assist the it with some speed- and efficiency-insensitive controls and computations.
- Power sensitive scenario: When the HP CPU is in the power-down state to save power, the LP CPU can be woken up to handle some external wake-up events.

Figure 3.1-1 shows the resources accessible to the HP CPU and the LP CPU.

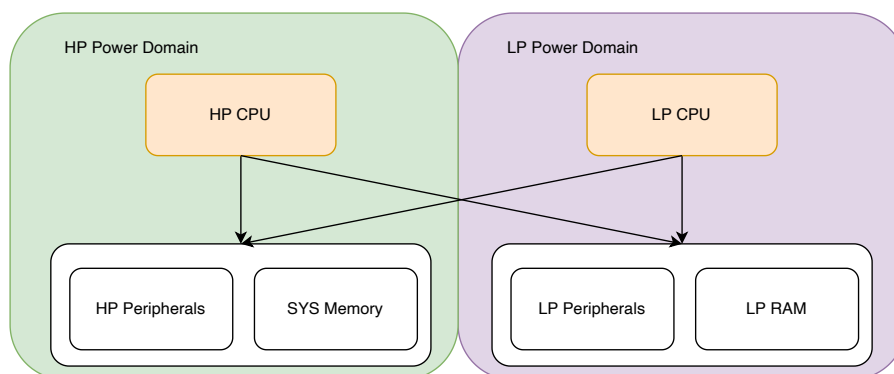


Figure 3.1-1. LP CPU Overview

In the figure above, the HP Power Domain and LP Power Domain can be powered on and off independently. When one domain is powered down, the CPU of the other domain cannot access the resources of that domain. In other words, only when the domain is powered on can its resources be accessed. For more information about power, please refer to the chapter 14 *Low-Power Management*.

3.2 Features

The LP CPU has the following features:

- Operating clock frequency of up to 40 MHz
- 18 vector interrupts
- Debug module compliant with RISC-V External Debug Support Version 0.13 with external debugger support over an industry-standard JTAG/USB port
- Hardware trigger compliant with RISC-V External Debug Support Version 0.13 with up to 2 breakpoints/watchpoints
- Core performance metric events
- Wake-up interrupt for HP CPU
- Access to HP memory and LP memory
- Access to the entire peripheral address space

3.3 Configuration and Status Registers (CSRs)

3.3.1 Register Summary

Below is a list of CSRs that are supported by the LP CPU. Except for the custom performance counter CSRs, all the implemented CSRs follow the standard mapping of bit fields as described in the RISC-V Instruction Set Manual, Volume II: Privileged Architecture, Version 1.10. It must be noted that even among the standard CSRs, not all bit fields have been implemented, limited by the subset of features implemented in the CPU. For a detailed description of the subset of fields implemented under each of these CSRs, please refer to Section 3.3.2.

The abbreviations given in Column **Access** are explained in Section [Access Types for Registers](#).

Name	Description	Address	Access
Machine Information CSR			
mhartid	Machine Hart ID	0xF14	RO
Machine Trap Setup CSRs			
mstatus	Machine Mode Status	0x300	R/W
misa ¹	Machine ISA	0x301	R/W
mie	Machine Interrupt Enable	0x304	R/W
mtvec ²	Machine Trap Vector	0x305	R/W
Machine Trap Handling CSRs			
mscratch	Machine Scratch	0x340	R/W
mepc	Machine Trap Program Counter	0x341	R/W
mcause ³	Machine Trap Cause	0x342	R/W
mtval	Machine Trap Value	0x343	R/W
mip	Machine Interrupt Pending	0x344	R/W
Trigger Module CSRs (shared with Debug Mode)			
tselect	Trigger Select Register	0x7A0	R/W

¹Although [misa](#) is specified as having both read and write access (R/W), its fields are hardwired and thus write has no effect. This is what would be termed WARL (Write Any Read Legal) in RISC-V terminology.

²[mtvec](#) only provides configuration for trap handling in vectored mode with the base address aligned to 256 bytes.

³External interrupt IDs reflected in [mcause](#) include even those IDs which have been reserved by RISC-V standard for core internal sources.

Name	Description	Address	Access
tdata1	Trigger Abstract Data 1	0x7A1	R/W
tdata2	Trigger Abstract Data 2	0x7A2	R/W
Debug Mode CSRs			
dcsr	Debug Control and Status	0x7B0	R/W
dpc	Debug PC	0x7B1	R/W
dscratch0	Debug Scratch Register 0	0x7B2	R/W
dscratch1	Debug Scratch Register 1	0x7B3	R/W
Machine Counter/Timer CSRs			
mcycle	Machine Clock Cycle Counter	0xB00	R/W
minstret	Machine Retired Instruction Counter	0xB02	R/W
mhpmcounter n(n:3-12)	Machine Performance Monitor Counter	0xB00+ n	R/W
mcycleh	The higher 32 bits of mcycle	0xB80	R/W
minstreth	The higher 32 bits of minstret	0xB82	R/W
mhpmcounter nh(n:3-12)	The higher 32 bits of mhpmcounter n(n:3-12)	0xB80+ n	R/W
Machine Counter Setup CSR			
mcountinhibit	Machine Counter Control	0x320	R/W

Note that if write, set, or clear operation is attempted on any of the read-only (RO) CSRs indicated in the above table, the CPU will generate an illegal instruction exception.

3.3.2 Registers

For how to program reserved fields, please refer to Section [Programming Reserved Register Field](#).

Register 3.1. mhartid (0xF14)

MHARTID	
31	0
0x00000000	
Reset	

MHARTID Represents hart ID. The LP CPU hart ID is 0. (RO)

Register 3.2. mstatus (0x300)

(reserved)				(reserved)				(reserved)				Mpp				(reserved)				MPIE				(reserved)				MIE				(reserved)			
31					22	21	20					13	12	11	10					8	7	6					4	3	2					0	
0x000					0	0x00					0x0					0x0					0	0x0					0	0x0					Reset		

- MIE** Write 1 to enable the global machine mode interrupt. (R/W)
- MPIE** Write 1 to enable the machine previous interrupt (before trap). (R/W)
- MPP** Configures machine previous privilege mode (before trap).
0x3: Machine mode
Other values: Invalid
Note: Only the lower bit is writable. Any write to the higher bit is ignored as it is directly tied to the lower bit.
(R/W)

Register 3.3. misa (0x301)

MXL		(reserved)				Z	Y	X	W	V	U	T	S	R	Q	P	O	N	M	L	K	J	I	H	G	F	E	D	C	B	A
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0x1		0x0		0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	1	0	0	0	0	0	1	0	1

Reset

MXL Machine XLEN = 1 (32-bit). (RO)**Z** Reserved = 0. (RO)**Y** Reserved = 0. (RO)**X** Non-standard extensions present = 0. (RO)**W** Reserved = 0. (RO)**V** Reserved = 0. (RO)**U** User mode implemented = 0. (RO)**T** Reserved = 0. (RO)**S** Supervisor mode implemented = 0. (RO)**R** Reserved = 0. (RO)**Q** Quad-precision floating-point extension = 0. (RO)**P** Reserved = 0. (RO)**O** Reserved = 0. (RO)**N** User-level interrupts supported = 0. (RO)**M** Integer Multiply/Divide extension = 1. (RO)**L** Reserved = 0. (RO)**K** Reserved = 0. (RO)**J** Reserved = 0. (RO)**I** RV32I base ISA = 1. (RO)**H** Hypervisor extension = 0. (RO)**G** Additional standard extensions present = 0. (RO)**F** Single-precision floating-point extension = 0. (RO)**E** RV32E base ISA = 0. (RO)**D** Double-precision floating-point extension = 0. (RO)**C** Compressed Extension = 1. (RO)**B** Reserved = 0. (RO)**A** Atomic Standard Extension = 1. (RO)

(reserved)
HP_IE
LP_RTC_IE
LP_WDT_IE
LP_TIMER_IE
MB_LP_IE
PMU_LP_IE
LP_ANAPER1_IE
LP_SYSREG_IE
LP_EFUSE_IE
LP_TSENS_IE
LP_TOUCH_IE
LP_ADC_IE
LP_GPIO_IE
LP_I2C_IE
(reserved)

(reserved)

LP_SPI_IE
(reserved)
LP_UART_IE
(reserved)
LP_SW_IE
(reserved)

Reset

HP_IE Write 1 to enable HP interrupt. (R/W)

BASE (reserved) MODE

Reset

BASE Configures the higher 24 bits of trap vector base address aligned to 256 bytes. (R/W)

Register 3.6. mscratch (0x340)

<i>MSCRATCH</i>	
31	0
0x00000000	
Reset	

MSCRATCH Contains machine scratch information for custom use. (R/W)

Register 3.7. mepc (0x341)

<i>MEPC</i>	
31	0
0x00000000	
Reset	

MEPC Configures the machine trap/exception program counter. This is automatically updated with address of the instruction which was about to be executed while CPU encountered the most recent trap. (R/W)

Register 3.8. mcause (0x342)

Interrupt Flag		(reserved)		Exception Code	
31	30	5	4	0	
0	0x00000000			0x00	Reset

Exception Code This field is automatically updated with unique ID of the most recent exception or interrupt due to which CPU entered trap. Possible exception IDs are:

0x2: Illegal instruction

0x3: Hardware breakpoint/watchpoint or EBREAK

0x6: Misaligned atomic instructions

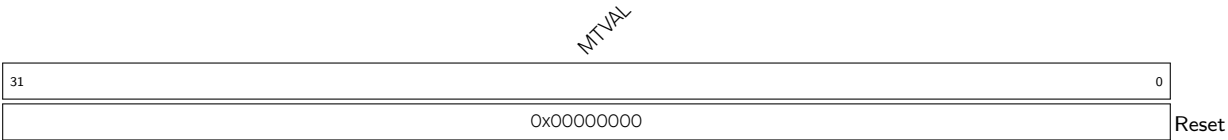
Note: Exception ID 0x0 (instruction access misaligned) is not present because CPU always masks the lowest bit of the address during instruction fetch.

(R/W)

Interrupt Flag This flag is automatically updated when CPU enters trap. If this is found to be set, it indicates that the latest trap occurred due to an interrupt. For exceptions it remains unset.

(R/W)

Register 3.9. mtval (0x343)



MTVAL Configures machine trap value. This is automatically updated with an exception dependent data which may be useful for handling that exception. Data is to be interpreted depending upon exception IDs:

- 0x1: Faulting address of instruction
- 0x2: Faulting instruction opcode
- 0x5: Faulting data address of load operation
- 0x7: Faulting data address of store operation

Note: The value of this register is not valid for other exception IDs and interrupts.

(R/W)

Register 3.10. mip (0x344)

(reserved)		HP_IP		LP_RTC_IP		LP_WDT_IP		LP_TIMER_IP		MB_LP_IP		PMU_IP		LP		ANAPERI_IP		LP_SYSREG_IP		LP_EFUSE_IP		LP_TSENS_IP		LP_TOUCH_IP		LP_ADC_IP		LP_GPIO_IP		LP_I2C_IP		(reserved)		(reserved)		LP_SPI_IP		(reserved)		LP_UART_IP		(reserved)		LP_SW_IP		(reserved)	
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15			12	11	10	8		7	6	4		3	2	0																	
0x0	0x0	0x0	0x0	0x0	0x0	0x0	0x0	0x0	0x0	0x0	0x0	0x0	0x0	0x0	0x0	0x0	0x0		0x0	0x0	0x0	0x0	0x0	0x0	0x0	0x0	0x0	0x0	0x0	0x0	Reset																

Reset

LP_SW_IP Configures the pending status of the LP software interrupt.

0 Not pending

1 Pending

(R/W)

LP_UART_IP Configures the pending status of the LP UART interrupt.

0 Not pending

1 Pending

(R/W)

LP_SPI_IP Configures the pending status of the LP SPI interrupt.

0 Not pending

1 Pending

(R/W)

LP_I2C_IP Configures the pending status of the LP I2C interrupt.

0 Not pending

1 Pending

(R/W)

LP_GPIO_IP Configures the pending status of the LP GPIO interrupt.

0 Not pending

1 Pending

(R/W)

LP_ADC_IP Configures the pending status of the LP ADC interrupt.

0 Not pending

1 Pending

(R/W)

LP_TOUCH_IP Configures the pending status of the LP TOUCH interrupt.

0 Not pending

1 Pending

(R/W)

LP_TSENS_IP Configures the pending status of the LP TSENS interrupt.

0 Not pending

1 Pending

(R/W)

LP_EFUSE_IP Configures the pending status of the LP eFuse interrupt.

0 Not pending

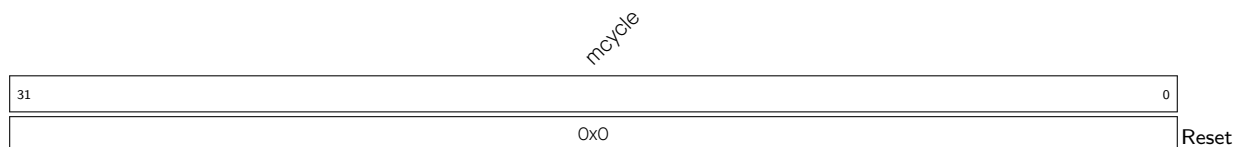
1 Pending

(R/W)

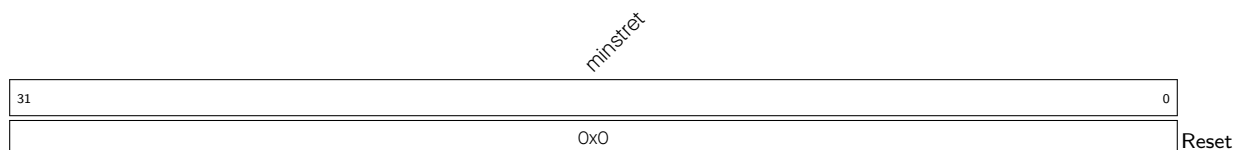
LP_SYSREG_IP Configures the pending status of the LP SYSREG interrupt.

0 Not pending

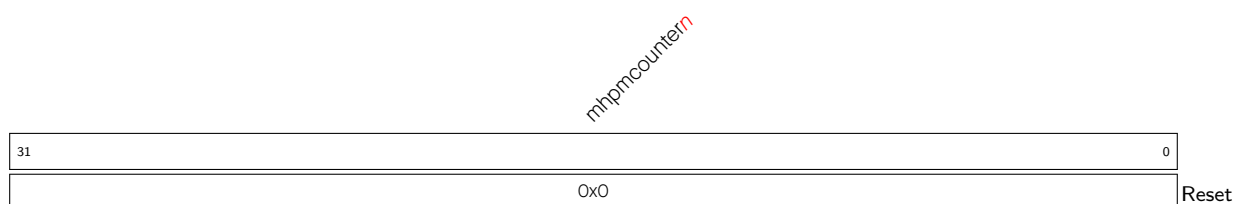
1 Pending

Register 3.11. mcycle (0xB00)

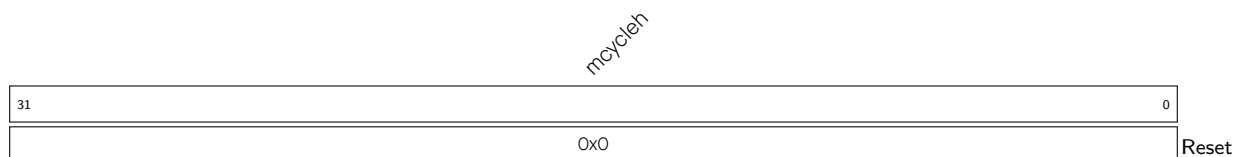
MCYCLE Configures the lower 32 bits of the clock cycle counter. (R/W)

Register 3.12. minstret (0xB02)

MINSTRET Configures the lower 32 bits of the instruction counter. (R/W)

Register 3.13. mhpmpcounter_{*n*} (*n*: 3-12) (0xB00+*n*)

MHPMCOUNTER_{*n*} Configures the lower 32 bits of the performance counter *n*. (R/W)

Register 3.14. mcycleh (0xB80)

MCYCLEH Configures the higher 32 bits of the clock cycle counter. (R/W)

Register 3.15. minstreth (0xB82)

minstreth	
31	0
0x0	
Reset	

MINSTRETH Configures the higher 32 bits of the instruction counter. (R/W)

Register 3.16. mhpmpcounter n (n : 3-12)h (0xB80+ n)

mhpmpcounter n	
31	0
0x0	
Reset	

MHPMCOUNTER n h Configures the higher 32 bits of the performance counter n . (R/W)

Register 3.17. mcountinhibit (0x320)

HPM		IR (reserved) CY		
		3	2	1 0
31	0	0x0	0x0	0x0
0x0		Reset		

HPM Configures whether the performance counter n (n :3-12) increments.

0: The counter does not count

1: The counter increments

(R/W)

IR Configure whether the instruction counter increments.

0: The counter does not count

1: The counter increments

(R/W)

CY Configure whether the clock cycle counter increments.

0: The counter does not count

1: The counter increments

(R/W)

3.4 Interrupts and Exceptions

The LP CPU handles interrupts and exceptions according to RISC-V Instruction Set Manual, Volume II: Privileged Architecture, Version 1.10. After entering an interrupt/exception handler, the CPU:

- Saves the current program counter (PC) value to the [mepc](#) CSR
- Copies the state of [MIE](#) of [mstatus](#) to [MPIE](#) of [mstatus](#)
- Saves the current privileged mode to [MPP](#) of [mstatus](#)
- Clears [MIE](#) of [mstatus](#)
- Toggles the privileged mode to machine mode (M mode)
- Jumps to the handler address
 - For exceptions, the handler address is the base address of the vector table in the [mtvec](#) CSR
 - For interrupts, the handler address is $mtvec + 4 * ID$, where ID is the interrupt ID. For more information, see Section 3.4.1
- After the mret instruction is executed, the core jumps to the PC saved in the [mepc](#) CSR, restores the value of [MPIE](#) of [mstatus](#) to [MIE](#) of [mstatus](#), and restores the privileged mode to that indicated by [MPP](#) of [mstatus](#)

When the core starts up, the base address of the vector table is initialized to the boot address 0x50000000. After startup, the base address can be changed by writing to the [mtvec](#) CSR. For more information about CSRs, see Section 3.3.1.

After a reset, the core starts to fetch instructions from the memory address specified by the LP_SYS_LP_CORE_BOOT_ADDR_REG register with an offset of 0x80. The default value for this register is 0x50100000, which corresponds to the starting address of the LP ROM.

3.4.1 Interrupts

The LP CPU in the ESP32-P4 supports 18 interrupt inputs. Each interrupt corresponds to an ID, which is denoted as *n* (where *n* = 3, 7, 11, 16 to 30), and has a specific entry address of $mtvec + 4 * n$. These interrupts are associated with specific peripheral sources. For the mapping between peripheral sources and interrupt IDs, please refer to 3.4-1.

Table 3.4-1. LP CPU Interrupt Mapping

Interrupt ID	Peripheral Source
3	LP_SW_INTR
7	LP_UART_INTR
11	LP_SPI_INTR
17	LP_I2C_INTR
18	LP_GPIO_INTR
19	LP_ADC_INTR
20	LP_TOUCH_INTR
21	LP_TSENS_INTR
22	LP_EFUSE_INTR

Interrupt ID	Peripheral Source
23	LP_SYSREG_INTR
24	LP_ANAPERI_INTR
25	PMU_REG_O_INTR, PMU_REG_1_INTR
26	MB_HP_INTR, MB_LP_INTR
27	LP_TIMER_REG_O_INTR, LP_TIMER_REG_1_INTR
28	LP_WDT_INTR
29	LP_RTC_INTR
30	HP_INTR

The read-only register LPINTR_STATUS_REG indicates the current status of the LP CPU external interrupt sources. For the mapping between this register and the interrupt sources, please refer to the description of this register.

3.4.2 Interrupt Handling

By default, interrupts are disabled globally because the MIE bit in `mstatus` has a reset value of 0. Software must set this bit to enable global interrupts.

Assuming the interrupt to enable is Interrupt `n`:

1. Enable Interrupt `n`:
 - To enable interrupts globally, write 1 to the MIE bit of `mstatus`.
 - To enable Interrupt `n`, write 1 to the `n`th bit of `mie` CSR.
2. After Interrupt `n` is enabled, the LP CPU can respond to it. Meanwhile, the corresponding peripheral interrupt should be configured to send an interrupt signal to the LP CPU.
3. After the interrupt is triggered, the LP CPU jumps to the $mtvec + 4 * n$ address.
4. To clear Interrupt `n`, the peripheral needs to clear the interrupt signal.

3.4.3 Exceptions

The LP CPU supports the RISC-V standard exceptions and can trigger the following exceptions:

Table 3.4-2. LP CPU Exception Causes

Exception ID	Description
1	Instruction access fault
2	Illegal instructions
3	Breakpoints (EBREAK)
5	Load access fault
6	Misaligned atomic instructions
7	Store access fault

3.5 Debugging

This section describes how to debug and test the LP CPU. Debug support is provided through standard JTAG pins and complies with RISC-V External Debug Support Version 0.13.

For ESP32-P4 system debugging overview, please refer to Chapter 1 [High-Performance CPU](#) > Figure 1.11-1 [Debug System Overview](#).

The user interacts with the Debug Host (e.g., laptop), which is running a debugger (e.g., gdb). The debugger communicates with a Debug Translator (e.g., OpenOCD, which may include a hardware driver) to communicate with Debug Transport Hardware (e.g., ESP-Prog adapter). The Debug Transport Hardware connects the Debug Host to the CPU's Debug Transport Module (DTM) through a standard JTAG interface. The DTM provides access to the debug module (DM) using the Debug Module Interface (DMI).

ESP32-P4 features two DTMs interconnected in a daisy-chain configuration. Specifically, TAP0 connects to the HP CPU, while TAP1 is dedicated to the LP CPU.

The HP DM supports multicore debugging according to the specification RISC-V External Debug Support Version 0.13. It can simultaneously control both cores of the HP CPU. In contrast, the LP DM is designed for debugging the single-core LP CPU.

The LP CPU implements four registers for core debugging: [dcsr](#), [dpc](#), [dscratch0](#), and [dscratch1](#). All of those registers can only be accessed from debug mode. If software attempts to access them when the LP CPU is not in debug mode, an illegal instruction exception will be triggered.

3.5.1 Features

The Low-Power CPU debug module has the following debugging features:

- Access to necessary information about the implementation
- CPU core halt and resume
- Read and write of CPU core registers (including CSRs)
- CPU core reset
- CPU halted on software breakpoint (planted breakpoint instruction)
- Hardware single-stepping
- Two hardware triggers (which can be used as breakpoints/watchpoints). See Section 3.6 for details

3.5.2 Functional Description

The debugging mechanism adheres to the specification RISC-V External Debug Support Version 0.13. For a detailed description of the debugging features, refer to the specification.

According to the specification, a hart has the following states: nonexistent, unavail, running, and halted. By default, the LP CPU is in the unavail state. To connect the LP CPU for debugging, users need to clear that state by configuring the [LPPERI_CPU_REG](#) register.

3.5.3 Register Summary

The following table lists the debug CSRs supported for the LP CPU.

The abbreviations given in Column **Access** are explained in Section [Access Types for Registers](#).

Name	Description	Address	Access
dcsr	Debug Control and Status	0x7B0	R/W
dpc	Debug PC	0x7B1	R/W
dscratch0	Debug Scratch Register 0	0x7B2	R/W
dscratch1	Debug Scratch Register 1	0x7B3	R/W

All debug module registers are implemented in accordance with the specification RISC-V External Debug Support Version 0.13. For more information, refer to the specification.

3.5.4 Registers

The following is a detailed description of the debug CSR supported by the LP CPU.

For how to program reserved fields, please refer to Section [Programming Reserved Register Field](#).

Register 3.18. dcsr (0x7B0)

xdebugver				(reserved)								ebreakm		(reserved)		ebreaku		(reserved)		cause			(reserved)		step		prv
31	28	27									16	15	14	13	12	11		9	8		6	5		3	2	1	0
0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1

Reset

xdebugver Represents the debug version.

4: External debug support exists
(RO)

ebreakm Configures execution of the EBREAK instruction in machine mode.

0: Trigger an exception with mcause = 3
1: Enter debug mode
(R/W)

ebreaku Configures execution of the EBREAK instruction in user mode.

0: Trigger an exception with mcause = 3 as described in privileged mode
1: Enter debug mode
(R/W)

cause Represents the reason why debug mode was entered. When there are multiple reasons to enter debug mode in a single cycle, the cause with the highest priority number is the one written.

1: An EBREAK instruction was executed. (priority 3)
2: The Trigger Module caused a halt. (priority 4)
3: haltreq was set. (priority 2)
4: The CPU core single stepped because step was set. (priority 1)
Other values: reserved for future use
(RO)

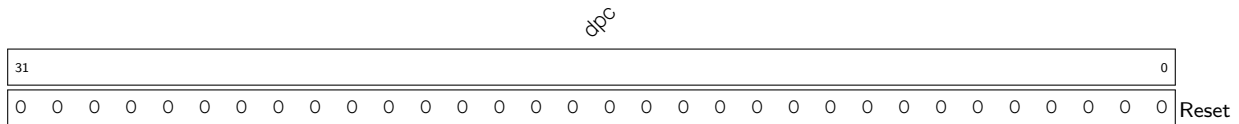
step When set and not in Debug Mode, the core will only execute a single instruction and then enter Debug Mode.

If the instruction does not complete due to an exception, the core will immediately enter Debug Mode before executing the trap handler, with appropriate exception registers set.

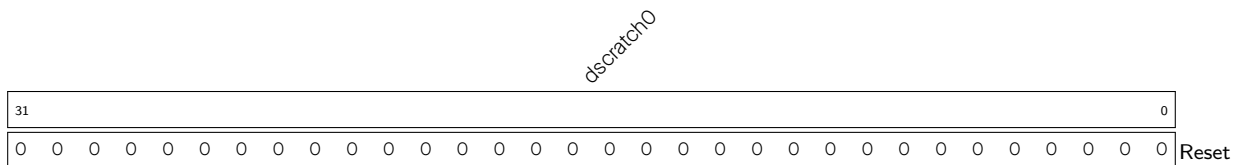
Setting this bit does not mask interrupts. This is a deviation from the RISC-V External Debug Support Specification Version 0.13.

(R/W)

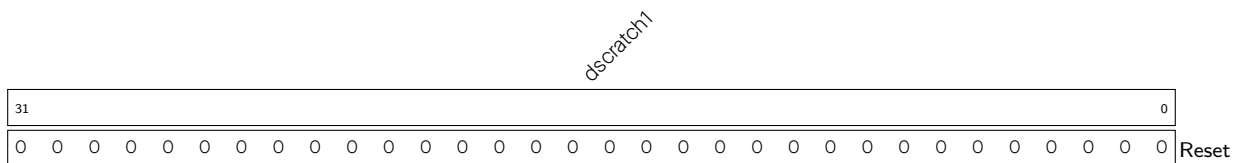
prv Contains the privilege level the core is operating in when debug mode is entered. A debugger can change this value to change the core's privilege level when exiting debug mode. Only **0x3** (machine mode) and **0x0** (user mode) are supported. (RO)

Register 3.19. dpc (0x7B1)

dpc Upon entry to debug mode, dpc is written with the address of the next instruction that will be executed. When resuming, the CPU core's PC is updated to the address stored in dpc. In debug mode, dpc can be modified. This field can be accessed in debug mode. (R/W)

Register 3.20. dscratch0 (0x7B2)

dscratch0 Used by the debug module internally. (R/W)

Register 3.21. dscratch1 (0x7B3)

dscratch1 Used by the debug module internally. (R/W)

3.6 Hardware Trigger

3.6.1 Features

Hardware Trigger module provides breakpoint and watchpoint capability for debugging. It has the following features:

- Two independent trigger units
- Configurable unit to match the address of the program counter
- Able to halt execution and transfer control to the debugger

3.6.2 Functional Description

The hardware trigger module provides three CSRs. See Section 3.6 for details. Among them, `tdata1` and `tdata2` are abstract CSRs, which means they are shadow registers for accessing internal registers in the trigger units, one at a time.

To select a specific trigger unit, the corresponding number (0-1) needs to be written to the [tselect](#) CSR. When a valid value is written, the abstract CSRs, [tdata1](#) and [tdata2](#), automatically match the internal registers of the trigger unit. Each trigger unit has two internal registers, namely [mcontrol](#) and [maddress](#), which are mapped to [tdata1](#) and [tdata2](#), respectively.

Writing a value more than 1 to [tselect](#) will set [tselect](#) to 1.

Since software or debugger may need to know the type of the selected trigger to correctly interpret [tdata1](#) and [tdata2](#), the 4-bit field (31-28) of [tdata1](#) encodes the type of the selected trigger. This type field is read-only and always provides a value of 0x2 for every trigger, which stands for support for address and data matching. Hence, it is inferred that [tdata1](#) and [tdata2](#) are to be interpreted as fields of [mcontrol](#) and [maddress](#), respectively. The specification RISC-V External Debug Support Version 0.13 provides information on other possible values, but the trigger module only supports the 0x2 type.

Once a trigger unit has been chosen by writing its index to [tselect](#), it will become possible to configure it by setting the appropriate bits in [mcontrol](#) CSR ([tdata1](#)) and writing the target address to [maddress](#) CSR ([tdata2](#)).

3.6.3 Trigger Execution Flow

When a hart is halted and enters debug mode due to the firing of a trigger ([action](#) = 1):

- [dpc](#) is set to the current PC in the decoding phase
- The cause field in [dcsr](#) is set to 2, which means halt due to trigger

3.6.4 Register Summary

Below is a list of Trigger Module CSRs supported by the CPU. These are only accessible from machine mode.

The abbreviations given in Column **Access** are explained in Section [Access Types for Registers](#).

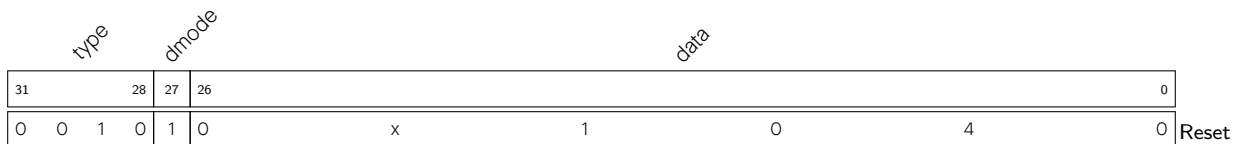
Name	Description	Address	Access
tselect	Trigger Select Register	0x7A0	R/W
tdata1	Trigger Abstract Data 1	0x7A1	R/W
mcontrol	tdata1 Shadow Register	0x7A1	R/W
tdata2	Trigger Abstract Data 2	0x7A2	R/W
maddress	tdata2 Shadow Register	0x7A2	R/W

3.6.5 Registers

For how to program reserved fields, please refer to Section [Programming Reserved Register Field](#).

Register 3.22. tselect (0x7A0)

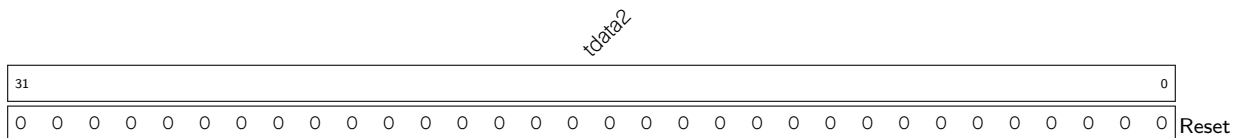
tselect Configures the index of the selected trigger unit. (R/W)

Register 3.23. tdata1 (0x7A1)

type Represents the trigger type. This field is reserved since only match type (0x2) triggers are supported. (RO)

dmode This is set to 1 if a trigger is being used by the debugger. This field is reserved since it is only supported in debug mode. (RO)

data Configures the abstract tdata1 content. This will always be interpreted as fields of [mcontrol](#) since only match type (0x2) triggers are supported. (R/W)

Register 3.24. tdata2 (0x7A2)

tdata2 Configures the abstract tdata2 content. This will always be interpreted as [maddress](#) since only match type (0x2) triggers are supported. (R/W)

Register 3.25. mcontrol (0x7A1)

(reserved)				dmode	maskmax				hit	select	timing	sizelo	action		chain	match	m	(reserved)			s	u	execute	store	load		
31	28	27	26			21	20	19	18	17	16	15		12	11	10		7	6	5	4	3	2	1	0		
0x2				0	0x0				0	0	0	0	0001		0	0	0	0	0	0	0	0	0	0	0		
Reset																											

Reset

dmode Same as [dmode](#) in [tdata1](#). (RO)

maskmax Represents the maximum NAPOT range.

0: A byte. Only exact match is supported.

Other values: Not supported.

(RO)

hit Not implemented in hardware. This field remains 0. (RO)

select Configures to select between an address match or a data match.

0: Perform a match on the virtual address

1: Perform a match on the data value loaded or stored, or the instruction executed

Note: Only address match is implemented. This field remains 0.

(RO)

timing Configures when the trigger will take action.

0: Take action before the instruction is executed

1: Take action after the instruction is executed

Note: The field remains 0.

(RO)

sizelo Only match of any size is supported. This field remains 0. (RO)

action Configure action of the selected trigger after it is triggered.

0x0: Cause a breakpoint exception

0x1: Enter debug mode (Valid only when dmode = 1)

Note: Only entering debug mode is supported. This field remains 1.

(RO)

CHAIN Not implemented in hardware. This field remains 0. (RO)

match Configures the trigger to perform the matching operation of the lower data/instruction address.

0x0: Exact match. Namely, the address corresponding to a certain byte during the access must exactly match the value of [maddress](#).

0x1: NAPOT match. Namely, at least one byte during the access is in the NAPOT region specified in [maddress](#).

Note: Only exact byte match is supported. This field remains 0.

(R/W)

m Set this field to make the selected trigger operate in machine mode. (RO)

s Set this field to make the selected trigger operate in supervisor mode. Operation in supervisor mode is not supported. This field is always 0. (RO)

Continued on the next page...

Register 3.25. mcontrol (0x7A1)**Continued from the previous page...**

u Set this field to make the selected trigger operate in user mode. Operation in user mode is not supported. This field is always 0. (RO)

execute Configures whether to enable the selected trigger to match the virtual address of instructions.

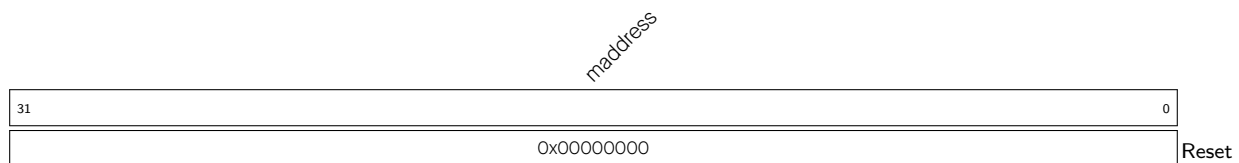
0: Not enable

1: Enable

(R/W)

store Set this field to make the selected trigger match the virtual address of the memory write operation. Not supported by hardware. This field is always 0. (RO)

load Set this field to make the selected trigger match the virtual address of a memory read operation. Not supported by hardware. This field is always 0. (RO)

Register 3.26. maddress (0x7A2)

maddress Configures the address used by the selected trigger when performing match operation. (R/W)

3.7 Performance Counter

The LP CPU implements one clock cycle counter [mcycle\(h\)](#), one instruction counter [minstret\(h\)](#), and ten event counters [mhpmcounter\(n:3-12\)](#). The clock cycle counter and instruction counter are always available and each is 64-bit wide. The performance counters are 40-bit wide each.

By default, all counters are enabled after reset. A counter can be enabled or disabled individually via the corresponding bit in the [mcountinhibit](#) CSR.

As shown in Table 3.7-1, each counter is dedicated to counting a particular event.

Table 3.7-1. Performance Counter

Counter	Counted Event
mcycle	Clock cycles
minstret	The number of instructions
mhpmcounter3	Wait cycles for memory access

Counter	Counted Event
mhpmcounter4	Wait cycles for fetching instructions
mhpmcounter5	The number of memory read operations. An unaligned read is counted as two.
mhpmcounter6	The number of memory write operations. An unaligned write is counted as two.
mhpmcounter7	The number of unconditional jump instructions (jal, jr, jalr)
mhpmcounter8	The number of branch instructions
mhpmcounter9	The number of taken branch instructions
mhpmcounter10	The number of compressed instructions
mhpmcounter11	Wait cycles for multiplication instructions
mhpmcounter12	Wait cycles for division instructions

3.8 System Access

The LP CPU has access to both memory and peripherals. When accessing an illegal address, i.e., outside the memory's or peripherals' address space, it will trigger the corresponding access error exception based on the type of access.

3.8.1 Memory Access

The ESP32-P4 LP CPU can access LP ROM, LP SRAM, and L2 SRAM. For more information, please refer to Section 7 [System and Memory](#).

- LP ROM: 16 KB region starting from 0x5010_0000 to 0x5010_3FFF. It is used for instruction fetch, data read, etc.
- LP SRAM: 32 KB region starting from 0x5010_8000 ~ 0x5010_FFFF. It is used for instruction fetch, data read, data write, etc.
- L2 SRAM: 768 KB region starting from 0x4FF0_0000 to 0x4FFB_FFFF. It is used for instruction fetch, data read, data write, etc.

Note:

The LP CPU experiences significant latency when accessing L2 SRAM, with each access taking approximately 20 LP CPU clock cycles. However, it can access LP ROM and LP SRAM with no latency.

The LP CPU supports the atomic instruction set. Both the LP CPU and the HP CPU can access memory through atomic instructions, thus achieving atomicity of memory access. For details on the atomic instruction set, please refer to RISC-V Instruction Set Manual Volume I: Unprivileged ISA, Version 2.2.

Note that only L2 SRAM region supports atomic access from HP CPU and LP CPU.

3.8.2 Peripheral Access

Table 7.3-2 in Chapter 7 [System and Memory](#) lists the peripherals accessible by the LP CPU and their base addresses.

3.9 Event Task Matrix Feature

The LP CPU on ESP32-P4 supports the Event Task Matrix (ETM) function, which allows LP CPU's ETM tasks to be triggered by any peripherals' ETM events, or LP CPU's ETM events to trigger any peripherals' ETM tasks. This section introduces the ETM tasks and events related to the LP CPU. For more information, please refer to [Chapter 13 Event Task Matrix \(ETM\)](#).

LP CPU can receive the following ETM task:

- ULP_TASK_WAKEUP_CPU: Wakes up the LP CPU.
- ULP_TASK_INT_CPU: Triggers an LP CPU interrupt.

LP CPU can generate the following ETM events:

- ULP_EVT_ERR_INTR: Indicates that an LP CPU exception occurs.
- ULP_EVT_START_INTR: Indicates that the LP CPU clock is turned on.
- ULP_EVT_HALT: Indicates that the LP CPU enters the sleep state

3.10 Sleep and Wake-Up Process

3.10.1 Features

Here are the sleep and wake-up features supported by the LP CPU.

- Independent operation, sleeping, and waking-up in the low-power system when the HP CPU is sleeping
- Actively configuring registers to enter the sleep state based on software operating status
- Multiple wake-up sources
- Sleep entry rejection by peripherals

3.10.2 Process

The LP CPU is in sleep by default and its wake-up module follows the process below to wake it up for work and make it sleep.

To configure wake-up sources, please refer to [Table 3.10-1](#).

The first startup of the LP CPU after power-up depends on the wake-up enable and wake-up source configuration by the HP CPU.

- Initialization of the LP CPU
 - Initialize the LP memory.
 - Start the LP CPU. Since the startup of the LP CPU depends on the wake-up process, it is recommended to use the [PMU_HP_TRIGGER_LP](#) register to start the initialization of the LP CPU in the following way:
 - * Set [PMU_LP_CPU_WAKEUP_EN](#) to 0x400000
 - * Set [PMU_HP_TRIGGER_LP](#) to 0x1

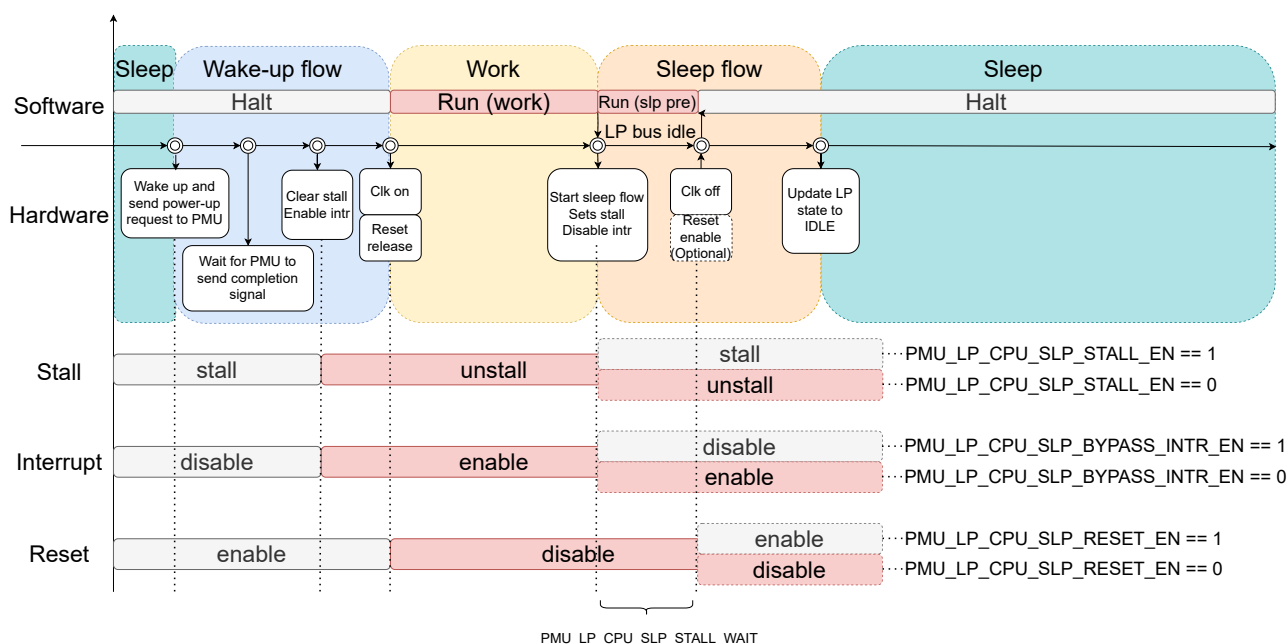


Figure 3.10-1. Wake-Up and Sleep Flow of LP CPU

* The LP CPU will go through the wake-up process to start running

- Wake-up process:

- The wake-up module receives a wake-up signal and sends a power-up request to the PMU.
- If the current power consumption state (clock, power supply, etc.) meets the requirements of the LP CPU, the PMU will immediately reply with the completion signal. Otherwise, it will adjust the power consumption state before replying with the completion signal.
- The wake-up module disables the STALL state of the LP CPU and enables interrupt receiving.
- The wake-up module starts the clock, releases reset (ignore this step if reset is not enabled for sleep), and starts working.

- Sleep process:

- The LP CPU configures the `PMU_LP_CPU_SLEEP_REQ` register to enable the wake-up module to start the sleep process.
- If `PMU_LP_CPU_SLP_STALL_EN` is 1, the wake-up module enables the STALL state of the LP CPU. If it is 0, the module does not enable that state. If `PMU_LP_CPU_SLP_BYPASS_INTR_EN` is 1, the module masks all the LP CPU's interrupts. If it is 0, the module does not mask them.
- The wake-up module waits for `PMU_LP_CPU_SLP_STALL_WAIT` LP CPU clock cycles, and then turns off the LP CPU clock. If `PMU_LP_CPU_SLP_RESET_EN` is 1, the module enables reset of the LP CPU.
- The wake-up module changes the state of the LP CPU to IDLE and the PMU can enter a deeper level of sleep state.

3.10.3 Wake-Up Sources

Table 3.10-1. Wake Sources

Register Value ¹	Wake-Up Source	Description
BIT(9)	LP IO	The LP CPU can be waken up by the LP IO interrupt status register signal. For more information, please refer to Chapter 9 <i>GPIO Matrix and IO MUX</i> .
BIT(10)	LP UART	The LP CPU can be waken up when the LP UART receives a certain number of RX pulses. REG_UART_WAKEUP_EN needs to be enabled. For more information, please refer to Chapter 42 <i>UART Controller (UART)</i> .
BIT(13)	RTC timer	The LP CPU can be waken up by the RTC timer target 0 timeout interrupt. For more information, please refer to Chapter 14 <i>Low-Power Management</i> .
BIT(14)	Brownout detection	The LP CPU can be waken up when a brownout occurs. For more information, please refer to Chapter 23 <i>Brownout Detector</i> .
BIT(17)	ETM	The LP CPU can be waken up by the ETM task. For more information, please refer to Chapter 13 <i>Event Task Matrix (ETM)</i> .
BIT(18)	RTC timer	The LP CPU can be waken up by the RTC timer target 0 timeout interrupt. For more information, please refer to Chapter 14 <i>Low-Power Management</i> .
BIT(19)	LP I2S audio	The LP CPU can be waken up when the predefined audio is received via I2S. For more details, please refer to Chapter 46 <i>I2S Controller (I2S)</i> .
BIT(22)	PMU_HP_TRIGGER_LP register	The HP CPU sets the register PMU_HP_TRIGGER_LP to wake up the LP CPU, and sets REG_HP_SW_TRIGGER_INT_CLR to clear this wake-up source.

¹ Value of the PMU_LP_CPU_WAKEUP_EN register

3.10.4 Sleep Rejection

When the LP CPU is preparing to enter the sleep state, it can be rejected by peripheral events. In other words, peripheral events have the capability to prevent the LP CPU from entering sleep mode. Below is the configuration process for the rejection:

1. The LP CPU configures PMU_SLP_REJECT_EN to enable the functionality that allows peripherals to reject the LP CPU from entering the sleep mode.
2. The LP CPU configures PMU_SLEEP_REJECT_ENA to enable the corresponding peripheral event to prevent the LP CPU from entering the sleep state.

The peripheral sources that can prevent the LP CPU from sleeping are the same as the wake-up sources.